# CS204 · DATA STRUCTURES & ALGORITHMS

Semester 4 syllabus and study material · 4 credits · 2026–27

**End-semester examination: 18 December 2026.** The exam is closed-book, 3 hours, and covers all five modules. Internal assessment includes two coding assignments and one mid-semester test. The syllabus explicitly rewards complexity analysis, correct edge-case handling, and the ability to choose an appropriate data structure.

## Course outcomes

• CO1 — Analyse the time and space complexity of iterative and recursive algorithms using asymptotic notation.

• CO2 — Implement linear structures (arrays, linked lists, stacks, queues) and select them for real problems.

• CO3 — Traverse, search, insert and delete in trees and graphs, including balanced-tree ideas.

• CO4 — Apply hashing, sorting and dynamic programming to solve optimisation problems.

• CO5 — Reason about correctness, trade-offs, and edge cases in a written examination.

**How this PDF is used in STUDYMATE AI:** this is a text-based PDF. The app extracts text page by page with a PDF parser, preserves page numbers for citations, chunks the content, embeds every chunk, and uses retrieval-augmented generation for questions. It does not send the whole document to the model for each question.

# Module 1 · Arrays, Strings, and Complexity

## 1.1 Arrays and memory

An array is a contiguous block of memory that stores elements of the same type. Because the address of the first element is known, the element at index *i* can be accessed in O(1) time: address = base + i × element_size. Insertion or deletion in the middle costs O(n) because later elements must shift. A dynamic array doubles its capacity when full, giving an amortised O(1) append cost.

## 1.2 Strings

A string is a sequence of characters. In most languages strings are immutable, so concatenating in a loop can cost O(n²). Use a character array or a builder instead. Important patterns: two pointers, sliding window, prefix sums, and frequency arrays. Always clarify whether the input alphabet is ASCII or Unicode, and whether case matters.

## 1.3 Asymptotic analysis

Big-O gives an upper bound on growth, Big-Omega a lower bound, and Big-Theta a tight bound. Drop constants and lower-order terms: 3n² + 5n + 9 is Θ(n²). Common growth rates from fastest to slowest: O(1), O(log n), O(n), O(n log n), O(n²), O(2■), O(n!). Master theorem: for T(n) = aT(n/b) + f(n), compare f(n) with n^(log_b a).

### Worked example

Find the largest sum of any contiguous subarray. Brute force checks all O(n²) subarrays. Kadane's algorithm keeps the best sum ending at the current position: current = max(a[i], current + a[i]); best = max(best, current). This is O(n) time and O(1) extra space, and works for arrays containing negative numbers.

# Module 2 · Linked Lists, Stacks, and Queues

## 2.1 Linked lists

A singly linked list stores each element in a node with a value and a next pointer. Insertion at the head is O(1); search is O(n); deleting a known node is O(1) if you have its predecessor. A doubly linked list also stores a previous pointer, enabling O(1) deletion with only the node reference. Linked lists avoid shifting but lose random access and have pointer overhead plus poor cache locality.

Each node stores data + a pointer to the next node. Access is sequential: O(n).

## 2.2 Stacks and queues

A stack is LIFO: push and pop are O(1). Applications include expression evaluation, balanced parentheses, undo, and iterative DFS. A queue is FIFO: enqueue at the rear and dequeue from the front. A circular array queue avoids O(n) shifts. A deque supports both ends in O(1). A monotonic stack finds the next greater element in O(n).

### Exam-style question

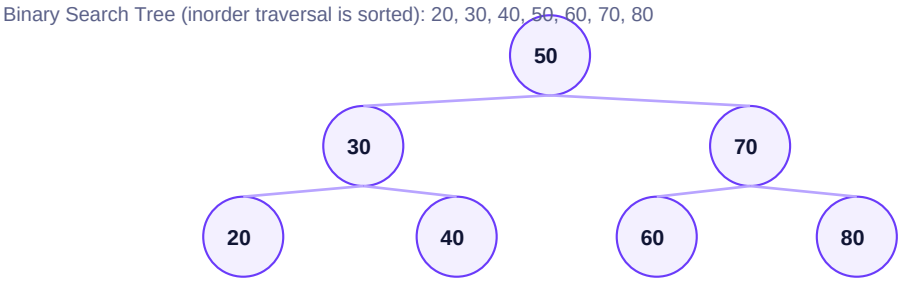
Q. Design a queue using two stacks and state the amortised cost of each operation. (6 marks)

Answer outline: keep an input stack and an output stack. Push is O(1). To dequeue, if the output stack is empty, pop every element from input and push it onto output; then pop output. Each element moves at most once, so the amortised dequeue cost is O(1), even though a single dequeue can be O(n).

# Module 3 · Trees, Heaps, and Balanced Search Trees

## 3.1 Binary trees and traversals

A binary tree has at most two children per node. Depth-first traversals are preorder (node, left, right), inorder (left, node, right), and postorder (left, right, node). In a binary search tree (BST), every left value is smaller and every right value is larger, so inorder traversal yields sorted order. Search, insert and delete are O(h), where h is height: O(log n) when balanced, O(n) in the worst case.




## 3.2 Heaps and priority queues

A binary heap is a complete tree stored in an array: children of index i are 2i+1 and 2i+2. A min-heap keeps the smallest key at the root. Insert and extract-min are O(log n); peek is O(1); building a heap bottom-up is O(n). Heap sort repeatedly extracts the root, giving O(n log n) time and O(1) extra array space. Priority queues are used in Dijkstra's algorithm and task schedulers.

## 3.3 Balanced trees

AVL trees maintain a balance factor in {-1, 0, 1} using rotations. Red-black trees relax balance but guarantee O(log n) height and are common in standard libraries. B-trees store many keys per node and are used for databases and file systems because they minimise disk reads.

# Module 4 · Graphs

## 4.1 Representations and traversal

Graphs can be stored as an adjacency list O(V+E) or an adjacency matrix O(V²). Breadth-first search uses a queue and finds shortest paths in unweighted graphs. Depth-first search uses recursion or a stack and supports cycle detection, topological sorting, and connected components. Always mark nodes as visited to avoid infinite loops.

## 4.2 Shortest paths and minimum spanning trees

Dijkstra's algorithm works for non-negative edge weights with a priority queue: O((V+E) log V). Bellman-Ford handles negative edges in O(VE) and detects negative cycles. Kruskal's MST algorithm sorts edges and uses union-find for cycle detection; Prim's algorithm grows a tree from a starting vertex. Topological sort is only defined for a directed acyclic graph (DAG).

# Module 5 · Hashing, Sorting, and Searching

## 5.1 Hash tables

A hash function maps a key to an index. A collision occurs when two keys map to the same index. Chaining stores a list per bucket; open addressing probes for the next free slot. With a good hash function and load factor below 1, average lookup is O(1); worst case is O(n). Load factor = number of entries / number of buckets. Rehashing restores a low load factor. Python's dict and Java's HashMap are hash tables.

## 5.2 Comparison sorting

Bubble, insertion, and selection sort are $O(n^2)$ and useful mainly for teaching or tiny inputs. Merge sort is stable, O(n log n), and uses O(n) auxiliary space. Quicksort is usually fastest in practice, O(n log n) average and $O(n^2)$ worst case; randomised pivots reduce that risk. Heapsort is O(n log n) in place but not stable. Comparison sorting has a lower bound of $\Omega(n \log n)$. Counting sort and radix sort can beat that bound when keys are small integers or fixed-length strings.

## 5.3 Binary search

Binary search requires a sorted array. Compare the target with the middle element and discard half the remaining range; complexity is O(log n). Use mid = low + (high - low) // 2 to avoid integer overflow in fixed-width languages. The harder part is finding the correct invariant for 'first true' or 'last true' variants.

# Module 6 · Dynamic Programming and Greedy Methods

## 6.1 Dynamic programming

Dynamic programming applies when a problem has overlapping subproblems and optimal substructure. Define the state, the transition, the base case, and the answer extraction. Top-down memoisation caches recursive results; bottom-up tabulation fills a table iteratively and can often save memory. Classic problems: 0/1 knapsack, longest common subsequence, edit distance, coin change, and matrix chain multiplication.

## 6.2 Greedy vs dynamic programming

A greedy algorithm makes the locally best choice and never reconsiders. It is correct only when the problem has the greedy-choice property and optimal substructure. Activity selection and Huffman coding are greedy. When a local choice can block a better global solution, use dynamic programming instead. A common exam trap is claiming that the greedy coin-change algorithm is always optimal; it fails for coin sets such as {1, 3, 4} and target 6.

# Assessment and revision guidance

Internal assessment: 30%. End-semester examination: 70%. The written exam contains short-answer questions, complexity derivations, tracing questions, and one design question. Code is not executed during the exam; pseudocode with clear invariants and edge cases is expected.

| Component | Weight | Notes |
|---|---|---|
| Mid-semester test | 15% | Modules 1–3 |
| Coding assignment 1 | 7.5% | Arrays, strings, complexity |
| Coding assignment 2 | 7.5% | Graphs and dynamic programming |
| End-semester exam | 70% | All modules, closed book |

# High-yield exam points

• State complexity in terms of n, V and E, and say whether it is average or worst case.

• For every algorithm, mention the invariant and one edge case (empty input, one element, duplicates, disconnected graph).

• Know the difference between a stable sort and an in-place sort, and give one example of each.

• Be able to compare adjacency list and adjacency matrix for sparse and dense graphs.

• For dynamic programming, write the recurrence before writing code.

• Simulate a small example by hand; it catches off-by-one errors quickly.

Revision plan suggestion: spend the first 40% of available time on weak modules, 30% on active recall and past questions, 20% on timed practice, and 10% on a final error log. Revisit mistakes after 24 hours and again after one week — spaced repetition is more effective than rereading.